

LogPress 이번 주 진행 요약

Part A는 H2O 재현 단계에 진입했고, Part B는 팀장님 Git 내용을 확인해 Thunderbird와 Nezha 진행 상태를 정리했습니다.

Part A

Baseline 재현 마무리 단계
H2O 핵심 코드 작성 시작

Part B

Thunderbird 기반은 대부분 완료
Nezha는 전체 매핑 필요

M3 전환

A는 C 기술 지원으로 이동
B는 Nezha 평가셋 준비

이번 주 핵심 결론

- Thunderbird는 처음부터 다시 파싱하지 않아도 되는 상태입니다.
- Part A는 H2O만 정리하면 Baseline 단계 종료가 가능합니다.
- Part B의 핵심은 Nezha의 root cause를 실제 로그 줄과 연결하는 작업입니다.

Part A 진행 상황

완료한 내용

- StreamingLLM, SnapKV, ObservedAttention 재현을 진행했습니다.
- Thunderbird 1시간 window를 모델 입력용 chunk로 나누었습니다.
- 압축 후 어떤 로그 줄이 살아남았는지 KEEP / PARTIAL / DROP 형태로 확인했습니다.

이번 주 H2O 진행

- H2O의 핵심 아이디어를 heavy hitter + recent token 구조로 정리했습니다.
- hh_score를 누적해서 중요한 token을 고르는 흐름을 코드로 작성했습니다.
- cache가 커졌을 때 top-k token과 최근 token을 남기는 기본 구조까지 작성했습니다.

다음 작업

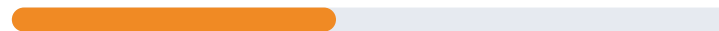
- 작은 tensor로 H2O 동작 검증
- Qwen의 KV head 구조 확인
- line-level 결과 형식으로 정리

Baseline 마무리 기준

StreamingLLM / SnapKV / ObservedAttention



H2O



Part B 인수인계 확인 결과

팀장님 Git에서 확인한 내용

- Thunderbird 전체 로그 파싱과 하드케이스 정리가 이미 상당 부분 진행되어 있었습니다.
- 원문 로그 복원과 킬스위치 실험용 데이터도 준비되어 있었습니다.
- 따라서 Part B가 Thunderbird를 처음부터 다시 처리할 필요는 낮습니다.

Nezha 현재 상태

- Nezha 샘플 CSV를 읽고, JSON 안의 message와 service를 뽑는 코드는 있습니다.
- 특정 원인 서비스를 직접 지정해서 원인 로그 줄을 찾는 실험도 되어 있습니다.
- 하지만 전체 장애 사례의 공식 원인을 자동으로 로그 줄에 연결하는 단계는 아직 남아 있습니다.

Part B 핵심 방향

Thunderbird 재작업보다 Nezha의 “장애 사례 → 공식 원인 → 실제 로그 줄” 연결을 우선 진행합니다.

M3 진입 후 작업 계획

Part A

- H2O 재현 마무리
- Baseline 결과 형식 통일
- Part C의 M3 구현 지원
- kvpress / Cache 구조 확인

Part B

- Nezha 전체 장애 사례 수 확인
- root cause 정보 위치 확인
- 원인 정보를 실제 로그 줄과 연결
- 개발용 / 평가용 데이터 분리

이번 회의에서 확인하면 좋은 질문

- 현재 확보된 Nezha 데이터가 전체 몇 개의 장애 사례인지 확인이 필요합니다.
- D가 사용한 Nezha 2개 사례가 전체 데이터인지, 테스트용 일부인지 확인이 필요합니다.
- M3에서 Part A는 구현 담당인지, C 지원 담당인지 역할을 다시 맞춰야 합니다.

요약

A는 H2O까지 Baseline을 마무리하고, B는 Nezha 원인 정보를 실제 로그 줄과 연결하는 작업에 집중합니다.

로그를 줄 단위 attention 으로 읽기

LogPress · 로그 입력부터 줄 단위 attention 까지

어느 로그 줄을 얼마나 참고했을까 ?

코드 파일	하는 일
token_mapper.py	토큰이 어느 로그 줄 · 필드에 속하는지 기록
attention.py	모델 실행 → 토큰 attention 추출 → 최종 평균
cross_line.py	토큰 attention 을 줄 × 줄로 묶기

토큰의 원래 로그 위치를 기록합니다

token_mapper.py · 모델 입력과 위치 정보를 함께 만듭니다

입력 문자열 만들기

timestamp → user → location
→ component → content

필드 값은 공백으로 연결하고
로그 줄 끝에는 개행을 넣습니다 .

line_id 와 label 은 모델 입력에 넣지 않습니다 .

예 : “disk error” 가 있는 줄

설명용 토큰	소속 줄	필드
disk	10	content
error	10	content
개행	10	없음

토큰의 글자 위치를 보고
원래 줄과 필드를 찾아 붙입니다 .

Q 는 참고하는 쪽 , K 는 참고받는 쪽

이미 계산된 attention 값을 , Q 방향과 K 방향으로 묶는 과정입니다 .

Q · 지금 정보를 찾는 토큰

예 : 11 번 줄의 토큰 t2

앞의 어떤 토큰을 얼마나
참고할지 가중치로 나타냅니다 .

**표의 한 행 =
Q 토큰 하나의 참조 내역**

K · 참고 대상이 되는 토큰

예 : 10 번 줄의 토큰 t0, t1

t2 가 t0 에 0.2, t1 에 0.3 을 주면
두 토큰을 그만큼 참고한 것입니다 .

표의 한 열 = K 토큰 하나

같은 로그 토큰도 행에서는 Q,
열에서는 K 역할을 합니다 .

토큰 attention 표를 읽어봅니다

attention.py · 행 = 참고하는 Query / 열 = 참고받는 Key

가상 예시 · 10 번 줄 : t0, t1 / 11 번 줄 : t2, t3

Query ↓ / Key →	t0 · 10 번	t1 · 10 번	t2 · 11 번	t3 · 11 번
t0 · 10 번	1.0	0	0	0
t1 · 10 번	0.4	0.6	0	0
t2 · 11 번	0.2	0.3	0.5	0
t3 · 11 번	0.1	0.2	0.3	0.4

예 : t2 는 t0 에 0.2, t1 에 0.3 의 attention 을 줍니다 .

K 는 같은 줄로 향한 값을 더합니다

토큰 하나하나에 준 attention 을 , 같은 로그 줄끼리 모읍니다 .

t2 가 10 번 줄을 참고한다면

10 번 줄의 토큰 t0 에 0.2
10 번 줄의 토큰 t1 에 0.3

두 값은 같은 줄로 향한 양입니다 .

줄 전체에 준 양은 합계

10 번 줄에 준 attention

$$0.2 + 0.3$$

$$= 0.5$$

K 까지 평균하면 0.25 가 되어 , “ 대상 토큰 하나당 값 ” 을 뜻하게 됩니다 .

Q 는 토큰 개수로 나눠 평균합니다

Q 토큰 수가 많아서 합계만 커지는 효과를 줄입니다 . 각 토큰이 0.4 씩 준다고 가정합니다 .

합계는 토큰 수에 따라 증가

Q 토큰 2 개인 줄
 $0.4 + 0.4 = 0.8$

Q 토큰 4 개인 줄
 $0.4 + 0.4 + 0.4 + 0.4 = 1.6$

**토큰마다 똑같이 참고해도
합계는 두 배가 됩니다 .**

평균은 토큰 하나당 값

토큰 2 개 : $0.8 \div 2 = 0.4$

토큰 4 개 : $1.6 \div 4 = 0.4$

토큰 하나가 평균적으로
얼마나 참고했는지 비교합니다 .

Q 쪽 개수 효과를 줄이는 것이며 ,
K 쪽 줄 길이 영향까지 없애지는 않습니다 .

계산 순서 : K 합산 $\rightarrow$ Q 합산 $\rightarrow$ Q 평균

11 번 줄의 Q 토큰 두 개가 10 번 줄에 준 값을 하나로 묶어봅니다 .

먼저 , 참고받는 줄로 합산

10 번 줄의 Key 는 t0, t1

$$t2: 0.2 + 0.3 = 0.5$$

$$t3: 0.1 + 0.2 = 0.3$$

그다음 , 참고하는 줄에서 평균

두 Q 토큰의 합계를 2 로 나눕니다 .

$$(0.5 + 0.3) \div 2$$

$$= 0.4$$

Key 줄 전체에 준 attention 을 , Query 토큰 1 개당 평균으로 표현합니다 .

토큰 4×4 를 로그 줄 2×2 로 묶습니다

cross_line.py · 이 변환 단계에서는 레이어와 헤드를 그대로 유지합니다

Query ↓ / Key →	10 번 줄	11 번 줄
10 번 줄	1.0	0.0
11 번 줄	0.4	0.6

0.4 = 11 번 줄의 토큰이 10 번 줄에 준 attention 의 평균

반환 형태 : [레이어 수 , 헤드 수 , Query 줄 수 , Key 줄 수]

레이어와 헤드를 평균해 최종 표를 만듭니다

여러 레이어 · 헤드에서 얻은 줄 × 줄 표를 , 최종 표 하나로 요약합니다 .

같은 위치의 값을 모아 평균

예 : 11 번 줄 → 10 번 줄

레이어 1 의 두 헤드 : 0.4, 0.6

레이어 2 의 두 헤드 : 0.2, 0.4

$$(0.4 + 0.6 + 0.2 + 0.4) \div 4 \\ = 0.4$$

출력값은 이렇게 읽습니다

11 번 줄이 10 번 줄을
평균적으로 얼마나 참고했는가

최종 출력 : 윈도우별 줄 × 줄 행렬

이후 단계 : 원인 분석 · KV 캐시 압축

LogPress 진행 보고

1 층 (Phase 1) — “ 모델이 원인 줄을 스스로 알아보나 ?” 를 확인하는 단계

- 1 확정 안 됐던 평가 항목 마무리
- 2 검증에 쓸 짝 데이터 확보 · 현재 상태
- 3 전체 파이프라인 → 보존율 결과

어텐션 = 모델이 한 줄을 읽을 때 다른 줄을 “얼마나 참고하는지” 의 정도

① 아직 확정 안 됐던 부분

빠져 있던 평가 지표를 정리하고, 애매했던 3 곳을 손봤다

빠져 있던 평가 지표 — 3 그룹

A. 미리 못 박을 것

필요한 짝 수 · 합격선 · 주지표 · 검정 설정

B. 검정 방법 세부

부트스트랩 · 여러 지표 볼 때 처리

C. 주장 방어용 비교

누수 통제 · ‘싼 신호’와 성능 대결

애매했던 3 곳 손볼 — 착시 점검

1

눈금 차이 착시

창마다 어텐션 총량이 달라 생기는 착시 → 총량 균형인지 점검

2

후보 범위 착시

등수를 5~15 번 줄로만 매김 → 빠진 줄의 영향 점검

3

짝 탈락 착시

로그 적은 서버는 짝이 탈락 → 양쪽 다 되는 짝만 비교

용어: Natural = 원본 그대로 (여러 서버 섞임, 주력) · Same-node = 같은 서버 로그만 모은 것 (대조용)

② 항목을 어떻게 채웠나 · 짹 데이터 확보

채움 현황

바로 확정

주지표 (등수차) · 순열검정 (5,000 · 양측) · 부트스트랩 (BCa) · 지표 1 개만 판정

데이터 나온 뒤

필요한 짹 수 (모의실험) · 합격선 (맞보기의 흠어짐 $\times 0.5$)

별도 분석

누수 통제 · ' 싹 신호 '와 성능 비교

합격선을 재려면 → 먼저 ' 짹 '이 필요

extract_pairs — 짹 뽑기 코드

원본 로그 2.1 억 줄에서 alert 1 개 ↔ normal 1 개를 조건 맞춰 짹지음

3,624

짹

729

종류

7

결측

다음 단계 — 받은 GPU 로 파이프라인 실행 (아직 시작 전)

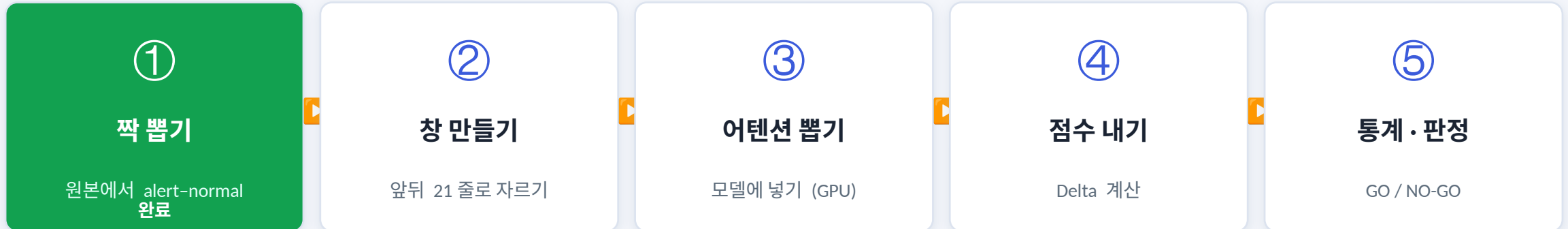
순서 : 창 만들기 → 어텐션 뽑기 → 점수 (Delta) 내기

이 Delta 로 → 파일럿 (합격선 확정) → 시물 (필요한 짹 수) 를 채운다 .

용어 : 짹 = 같은 서버 · 비슷한 시간 · 길이로 맞춘 alert-normal 한 쌍 / Delta = 두 줄의 신호 차이 (alert - normal)

용어 : alert = 로그 맨 앞칸에 이상 표시가 붙은 줄 (정상은 ' - ') / normal = 그 표시가 없는 평범한 줄

③ 전체 파이프라인 — 어디까지 왔나



여기까지 CPU(내 노트북) 로 가능

여기부터 GPU 필요

그럼 ‘보존율’과는 어떻게 이어지나 ?

이 파이프라인

“ 어텐션이 원인 줄을 알아보나 ? ”
= 신호가 있는지 확인하는 관문

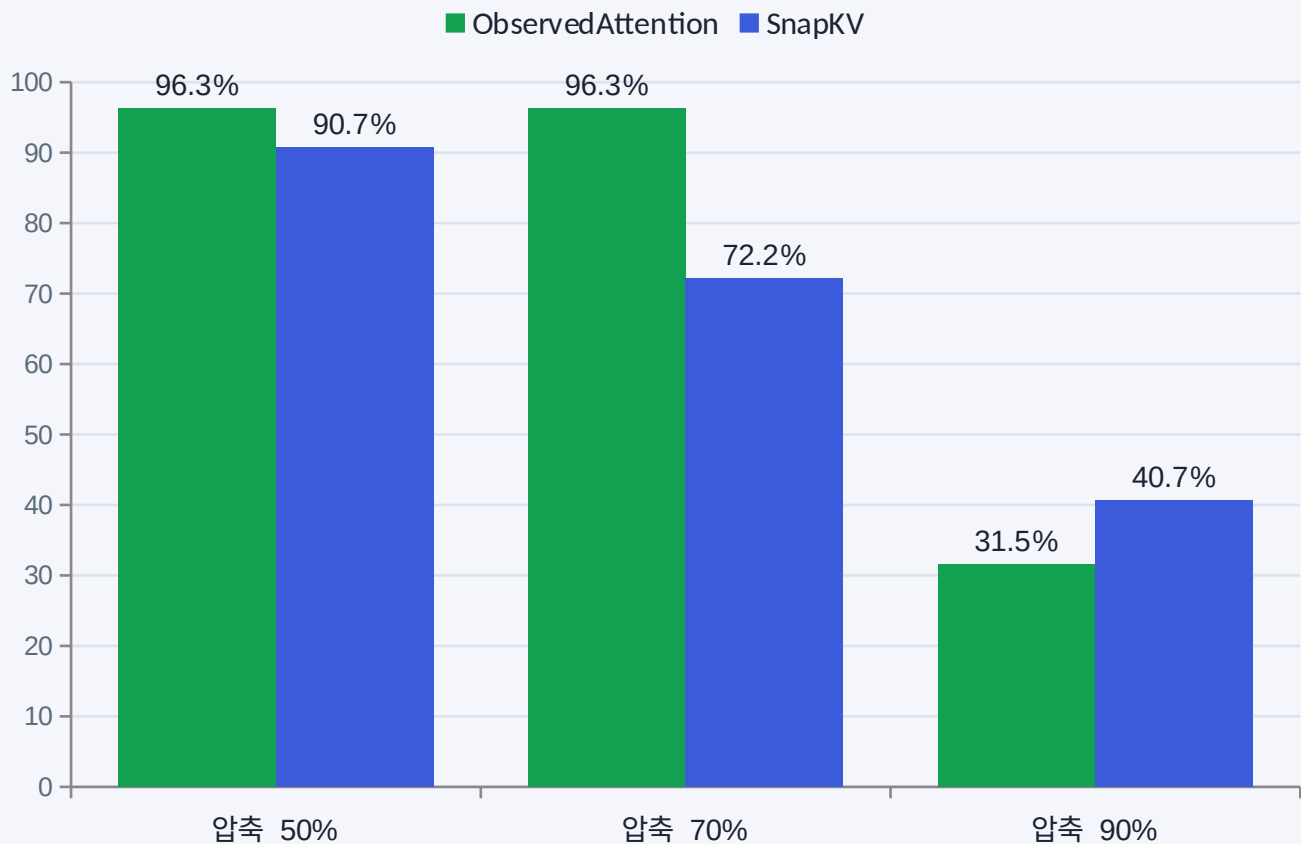
보존율

“ 압축해도 원인 줄이 살아남나 ? ”
= 압축 방식을 평가하는 것

→ 둘 다 핵심은 “ 원인 줄 ” . 신호가 있어야 (관문 통과) 압축이 원인을 지킬 이유가 생긴다.

보존율 결과 — chunk13 (baseline 비교)

원인 줄 (alert 54 개) 중 압축 후 살아남은 비율 · overall 기준



관찰

ObservedAttention

70% 까지 96% 로 버티다 90% 에서 급락 (31.5%). 절벽이 70~90% 사이 .

SnapKV

50→70→90% 로 갈수록 완만히 감소 (90.7 → 72.2 → 40.7%).

90% 에서 역전

아주 세게 압축하면 SnapKV 가 더 버팀 (40.7 > 31.5).

참고: non_error 층은 이 chunk 가 VAPI 한 종류뿐이라 항상 빔 (데이터 특성) · 0.6~0.8 압축률은 결과 파일에 없어 제외

용어: 압축률 = KV 를 얼마나 버리나 (90% 면 90% 버림) / 보존율 = 원인 줄 중 살아남은 비율